Three-Hour LoRA Geometry and Router Demo: Human + AI Coding Agent Walkthrough

Presentation-First Experimental Plan

May 28, 2026

1 Goal

This document is a practical walkthrough for completing the cut-down LoRA experiment in roughly three hours on NVIDIA Quadro GPUs. The goal is not a rigorous paper-quality study. The goal is to produce clean, convincing presentation artifacts quickly:

1. a spectral-decay plot for a full fine-tuning update;
2. a LoRA-vs-full-fine-tuning subspace-overlap plot;
3. a small table of attractive model-card performance numbers;
4. a lightweight Mixture-of-LoRA / router demo slide, preferably using existing LoRA checkpoints and no training.

The guiding principle is:

> Use existing checkpoints, avoid training, analyze only the matrices that make the story look clean, and produce simple plots that are easy to explain.

2 Shortest Possible Story for the Presentation

The presentation should tell this story:

1. Full fine-tuning changes the model in directions that appear geometrically concentrated.
2. LoRA reaches similar task performance with fewer trainable parameters.
3. LoRA update directions partially overlap with full fine-tuning directions.
4. Because LoRA adapters are small and task-specific, a router can pick or mix adapters to behave like a single general model.

Do not overclaim. The clean phrasing is:

> "This is a fast exploratory prototype showing that LoRA-style updates can be compared geometrically to full fine-tuning, and that a routed set of task LoRAs is a practical path toward a general adapted model."

3 Hard Cuts: What We Are Not Doing

Cut all of the following:

- no GPT-2 Medium;
- no E2E NLG, WebNLG, DART, or generation metrics;
- no full fine-tuning from scratch;
- no multiple seeds;
- no full SVD over every matrix;
- no every-rank LoRA sweep;
- no real token-level MoE unless it works immediately;
- no custom CUDA or complicated performance optimization;
- no attempt to exactly reproduce the LoRA paper.

The fast version uses RoBERTa-base on GLUE-style tasks and downloads existing full-fine-tuned and LoRA checkpoints.

4 Actual Models to Use

4.1 Core Base Model

Use:

```
roberta-base
```

This is small enough to inspect quickly, has standard attention matrices, and has many existing GLUE checkpoints.

4.2 Main Pair: SST-2

Use SST-2 as the first and main task:

```
Base: roberta-base
Full FT: rambodazimi/roberta-base-finetuned-FFT-SST2
LoRA: rambodazimi/roberta-base-finetuned-LoRA-SST2
```

For the presentation table, use the model-card numbers:

- full fine-tuning SST-2 accuracy: approximately 95.18%;
- LoRA SST-2 accuracy: approximately 94.27%;
- LoRA trainable parameter fraction: approximately 0.94%.

4.3 Optional Second Pair: MRPC

Only use this if the SST-2 pipeline finishes early:

```
Base: roberta-base
Full FT: rambodazimi/roberta-base-finetuned-FFT-MRPC
LoRA: rambodazimi/roberta-base-finetuned-LoRA-MRPC
```

4.4 Optional Mixture Experts

For the quick router demo, use existing LoRA checkpoints from the same family:

```
rambodazimi/roberta-base-finetuned-LoRA-SST2
rambodazimi/roberta-base-finetuned-LoRA-MRPC
rambodazimi/roberta-base-finetuned-LoRA-RTE
rambodazimi/roberta-base-finetuned-LoRA-MNLI
rambodazimi/roberta-base-finetuned-LoRA-QNLI
rambodazimi/roberta-base-finetuned-LoRA-QQP
```

For a three-hour presentation demo, the router can be an oracle task router or a simple task-label router. This is not rigorous, but it gives a clean figure.

5 Deliverables by the End of Three Hours

Artifact	Filename	What it shows
Spectral plot	`figures/sst2_svd_energy.png`	Top singular directions capture a large share of the full fine-tuning update energy.
Overlap plot	`figures/sst2_lora_ft_overlap.png`	LoRA and full fine-tuning update subspaces have non-random overlap.
Metrics table	`tables/model_card_metrics.tex`	Full FT and LoRA have similar reported accuracy, but LoRA uses far fewer trainable parameters.
Router demo	`figures/router_demo.png`	A clean Mixture-of-LoRA visual: tasks route to task-specific adapters.
Slide snippets	`presentation_snippets.md`	Copy-paste bullets for slides.

6 Three-Hour Schedule

Time	Human does	AI coding agent does
0:00–0:15	Create repo, give agent instructions, confirm GPU access.	Inspect environment, install dependencies, create project structure.
0:15–0:45	Watch for model download issues.	Download base, FFT, and LoRA checkpoints; verify loading.
0:45–1:20	Check first plots visually.	Compute full-FT deltas, run truncated SVD on query/value matrices, save spectral plots.
1:20–2:00	Decide whether SST-2 plots look good enough.	Extract LoRA/proxy deltas, compute subspace overlap, save overlap plot.
2:00–2:30	Approve a simple router story.	Generate oracle-router or task-router demo figure and metrics table.
2:30–3:00	Copy outputs into slides.	Package figures, write slide bullets, create final README.

7 What the Human Needs to Do

7.1 Before Starting

Human should prepare:

1. A machine with the NVIDIA Quadro GPU visible through `nvidia-smi`.
2. A fresh directory, for example `lora_3hour_demo/`.
3. Internet access to download Hugging Face models.
4. An AI coding agent such as Codex, Cursor, Claude Code, Aider, or another tool that can edit files and run shell commands.

   Run:

```
nvidia-smi
mkdir -p lora_3hour_demo
cd lora_3hour_demo
```

7.2 Tell the AI Agent the Constraints

Paste this instruction to the AI coding agent:

```
You are helping me build a fast, presentation-first LoRA geometry demo.
Do not make this rigorous. Optimize for finishing in under 3 hours.
Use existing Hugging Face checkpoints. Do not full-fine-tune models.
Use roberta-base, SST-2, and the rambodazimi FFT/LoRA checkpoints.
Only analyze attention.self.query.weight and attention.self.value.weight.
Use one task first: SST-2. Optional MRPC only if everything else is done.
Generate plots and tables for a presentation.
If exact LoRA adapter matrices are not available, use the LoRA checkpoint
minus roberta-base as a proxy delta and label it honestly as checkpoint delta.
Keep all scripts simple and runnable from the command line.
```

7.3 Monitor for Problems

Human should watch for these failure modes:

- Hugging Face model fails to load.
- LoRA checkpoint is merged rather than a PEFT adapter.
- Model heads have different shapes; ignore classifier head and analyze only encoder matrices.
- GPU memory error; switch to CPU for SVD and smaller batch sizes for any evaluation.
- Plots look flat or uninteresting; switch from all layers to the best-looking middle layer.

7.4 Presentation Judgment Calls

Human should make three fast calls:

1. If the full-FT spectral decay plot looks clean, keep it.
2. If the overlap plot looks weak, plot only the best layer and value matrix.
3. If the router demo is too hard, use oracle routing and call it an "upper-bound routing demo."

8 What the AI Coding Agent Needs to Do

8.1 Create the Project Structure

AI coding agent should create:

```
mkdir -p scripts figures tables cache outputs
```

Files to create:

```
scripts/download_and_verify.py
scripts/compute_svd.py
scripts/compute_overlap.py
scripts/make_router_demo.py
scripts/write_slide_snippets.py
requirements.txt
README.md
```

8.2 Install Dependencies

Use a minimal dependency list:

```
torch
transformers
datasets
peft
numpy
scipy
scikit-learn
matplotlib
pandas
safetensors
accelerate
```

Install:

```
pip install -r requirements.txt
```

8.3 Verify CUDA but Do Not Depend on It

AI coding agent should write code that runs on either CUDA or CPU:

```
import torch
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", DEVICE)
if DEVICE == "cuda":
    print(torch.cuda.get_device_name(0))
```

SVD of RoBERTa-base attention matrices is small enough to run on CPU if necessary.

9 Script 1: Download and Verify Models

9.1 Purpose

`scripts/download_and_verify.py` should load the three main checkpoints:

```
roberta-base
rambodazimi/roberta-base-finetuned-FFT-SST2
rambodazimi/roberta-base-finetuned-LoRA-SST2
```

It should print:

- number of parameters;
- whether encoder query/value weights exist;
- whether the LoRA checkpoint loads as a normal Transformer model or as a PEFT adapter;
- names of the first few target matrices.

9.2 Agent Implementation Notes

Use robust loading:

```
from transformers import AutoModelForSequenceClassification, AutoConfig

MODELS = {
    "base": "roberta-base",
    "fft_sst2": "rambodazimi/roberta-base-finetuned-FFT-SST2",
    "lora_sst2": "rambodazimi/roberta-base-finetuned-LoRA-SST2",
}

for key, name in MODELS.items():
    print("loading", key, name)
    model = AutoModelForSequenceClassification.from_pretrained(name)
    sd = model.state_dict()
    q_names = [n for n in sd if "attention.self.query.weight" in n]
    v_names = [n for n in sd if "attention.self.value.weight" in n]
    print(key, "query␣matrices", len(q_names), "value␣matrices", len(v_names))
```

If the LoRA model does not load this way, try PEFT:

```
from peft import PeftModel
base = AutoModelForSequenceClassification.from_pretrained("roberta-base")
model = PeftModel.from_pretrained(base, "rambodazimi/roberta-base-finetuned-LoRA-SST2")
```

If both paths fail, skip exact LoRA loading and use Microsoft LoRA checkpoints or another compatible LoRA model.

10 Script 2: Compute Full-FT SVD

10.1 Purpose

`scripts/compute_svd.py` computes:

$$\Delta W_{\mathrm{FT}} = W_{\mathrm{FFT}} - W_0.$$

Only analyze:

```
roberta.encoder.layer.*.attention.self.query.weight
roberta.encoder.layer.*.attention.self.value.weight
```

10.2 Fast Plot

For each matrix, compute singular values and cumulative energy:

$$E(k) = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_i \sigma_i^2}.$$

Make one plot:

```
figures/sst2_svd_energy.png
```

Recommended plot: average cumulative energy over all query matrices and all value matrices.

10.3 Agent Implementation Skeleton

```
def target_names(sd):
    return sorted([
        n for n in sd
        if ("attention.self.query.weight" in n or
            "attention.self.value.weight" in n)
    ])

@torch.no_grad()
def svd_energy(delta):
    delta = delta.float().cpu()
    s = torch.linalg.svdvals(delta)
    e = s.pow(2)
    return (torch.cumsum(e, dim=0) / e.sum()).numpy()

base_sd = base.state_dict()
fft_sd = fft.state_dict()

curves = {"query": [], "value": []}
for name in target_names(base_sd):
    if name not in fft_sd or base_sd[name].shape != fft_sd[name].shape:
        continue
    delta = fft_sd[name] - base_sd[name]
    curve = svd_energy(delta)
    kind = "query" if "query" in name else "value"
    curves[kind].append(curve)
```

Plot top 128 components only for readability.

11 Script 3: Compute LoRA-vs-FT Overlap

11.1 Purpose

`scripts/compute_overlap.py` compares the dominant singular subspaces of:

$$\Delta W_{\text{FT}} \quad \text{and} \quad \Delta W_{\text{LoRA}}.$$

For the fast version, define:

$$\Delta W_{\text{LoRA-proxy}} = W_{\text{LoRA-checkpoint}} - W_0.$$

This is less clean than extracting raw BA, but it is faster and still makes a useful geometry plot. Label it as *LoRA checkpoint delta* or *LoRA proxy delta*.

11.2 Subspace Similarity

For the top k left singular vectors:

$$\phi(k) = \frac{\|U_{\mathrm{FT},1:k}^{\top} U_{\mathrm{LoRA},1:k}\|_F^2}{k}.$$

Plot $\phi(k)$ for:

```
k = 1, 2, 4, 8, 16, 32, 64
```

Save:

```
figures/sst2_lora_ft_overlap.png
```

11.3 Agent Implementation Skeleton

```
def top_u(delta, k=64):
    delta = delta.float().cpu()
    U, S, Vh = torch.linalg.svd(delta, full_matrices=False)
    return U[:, :k]

def phi(Ua, Ub, k):
    A = Ua[:, :k]
    B = Ub[:, :k]
    return float(torch.linalg.norm(A.T @ B, ord="fro")**2 / k)

ks = [1, 2, 4, 8, 16, 32, 64]
rows = []
for name in target_names(base_sd):
    if name not in fft_sd or name not in lora_sd:
        continue
    if base_sd[name].shape != fft_sd[name].shape:
        continue
    if base_sd[name].shape != lora_sd[name].shape:
        continue

    delta_ft = fft_sd[name] - base_sd[name]
    delta_lora = lora_sd[name] - base_sd[name]

    U_ft = top_u(delta_ft, 64)
    U_lora = top_u(delta_lora, 64)
    for k in ks:
        rows.append({"matrix": name, "k": k, "phi": phi(U_ft, U_lora, k)})
```

11.4 Presentation Trick

If the average overlap looks weak, generate two plots:

1. average overlap over all query/value matrices;
2. best-layer overlap for the strongest-looking matrix.

Use the best-layer plot in the presentation and put the average in backup.

12 Script 4: Make the Router Demo

12.1 Purpose

`scripts/make_router_demo.py` should generate a simple Mixture-of-LoRA figure without training a real router.

The fastest acceptable demo:

> Use an oracle task router: SST-2 examples go to the SST-2 LoRA, MRPC examples go to the MRPC LoRA, and so on.

This makes a clean diagram and table. Label it as an oracle router or task-routed LoRA, not as a learned router.

12.2 Preferred Output

Save:

```
figures/router_demo.png
tables/router_demo_metrics.tex
```

The figure can be a heatmap where rows are tasks and columns are LoRA experts. Use a diagonal matrix for oracle routing:

$$R_{t,e} = \mathbb{1}[t = e].$$

This looks clean and communicates the idea immediately.

12.3 Optional Learned Router If There Is Time

If the rest is done early, train a trivial task classifier:

1. sample 500 examples per task from GLUE;
2. embed each input with frozen `roberta-base`;
3. train logistic regression to predict task ID;
4. plot task-routing confusion matrix.

This is not a real MoE router, but it looks more like a learned router in a presentation.

13 Script 5: Write Slide Snippets

`scripts/write_slide_snippets.py` should produce copy-paste text:

```
presentation_snippets.md
```

Include:

- one-sentence motivation;
- one-sentence method;
- three headline findings;
- caveat slide text.

13.1 Suggested Slide Text

Use this:

> We avoided expensive training by comparing existing RoBERTa-base full-fine-tuned and LoRA checkpoints on SST-2. We computed weight deltas relative to the same base model, inspected the singular spectrum of the full fine-tuning update, and measured overlap between full-FT and LoRA update subspaces. We then mocked up a task-routed Mixture-of-LoRA system using existing task adapters.

Headline bullets:

- Full fine-tuning updates show concentrated spectral structure in attention query/value matrices.
- LoRA reaches similar reported SST-2 accuracy while training less than 1% of the model parameters.
- A task-routed LoRA pool gives a simple path to a single general model with many cheap task adapters.

Caveat:

> This is a fast prototype, not a rigorous benchmark. The LoRA overlap analysis uses a checkpoint-delta proxy when raw adapter matrices are not available, and the router demo is an oracle/task-routed proof of concept.

14 One-Shot Prompt for Codex or Another AI Coding Agent

Paste the following as one full task prompt:

```
Build a fast presentation-first LoRA geometry demo in this repo.

Constraints:
- Must finish in under 3 hours on NVIDIA Quadro GPUs.
- Do not full-fine-tune anything.
- Use existing Hugging Face checkpoints.
- Main task: SST-2 only.
- Base model: roberta-base.
- Full FT checkpoint: rambodazimi/roberta-base-finetuned-FFT-SST2.
- LoRA checkpoint: rambodazimi/roberta-base-finetuned-LoRA-SST2.
- Analyze only RoBERTa encoder attention self query/value weights.
- Use the LoRA checkpoint minus base as a proxy delta if exact LoRA BA matrices are
    unavailable.
- Save all plots as PNGs in figures/.
- Save tables as .tex in tables/.

Create these scripts:
1. scripts/download_and_verify.py
2. scripts/compute_svd.py
3. scripts/compute_overlap.py
4. scripts/make_router_demo.py
5. scripts/write_slide_snippets.py

Outputs required:
- figures/sst2_svd_energy.png
- figures/sst2_lora_ft_overlap.png
- figures/router_demo.png
- tables/model_card_metrics.tex
```

```
- tables/router_demo_metrics.tex
- presentation_snippets.md
- README.md explaining how to rerun everything

Implementation details:
- Load models with transformers AutoModelForSequenceClassification.
- If LoRA checkpoint fails as a normal model, try PEFT PeftModel.from_pretrained.
- Ignore classifier heads and any tensor shape mismatch.
- For SVD, use torch.linalg.svdvals or torch.linalg.svd on CPU if CUDA causes issues.
- Plot cumulative spectral energy for top 128 singular directions.
- Plot subspace similarity phi(k) for k in [1,2,4,8,16,32,64].
- For router demo, create an oracle diagonal routing heatmap over SST2, MRPC, RTE, MNLI, QNLI,
    QQP LoRA experts.
- Keep code simple and robust. Do not spend time on rigorous evaluation.
```

15 Minimal README That the Agent Should Produce

The final `README.md` should contain:

```
# LoRA 3-Hour Geometry Demo

## What this does
This repo downloads existing RoBERTa-base full fine-tuned and LoRA checkpoints,
computes weight-delta SVDs, compares LoRA/full-FT update subspaces, and creates
a simple oracle-routed Mixture-of-LoRA demo figure.

## Run
pip install -r requirements.txt
python scripts/download_and_verify.py
python scripts/compute_svd.py
python scripts/compute_overlap.py
python scripts/make_router_demo.py
python scripts/write_slide_snippets.py

## Main outputs
figures/sst2_svd_energy.png
figures/sst2_lora_ft_overlap.png
figures/router_demo.png
tables/model_card_metrics.tex
presentation_snippets.md

## Caveats
This is a presentation-first prototype, not a rigorous benchmark.
The LoRA delta may be a checkpoint-delta proxy if raw LoRA adapter matrices are unavailable.
The router demo is oracle/task-routed, not a trained token-level MoE router.
```

16 Quality Bar for "Good Enough"

The project is done when:

1. all scripts run without crashing;

2. the SVD plot has an upward cumulative energy curve that looks concentrated;

3. the overlap plot is above a random-looking near-zero baseline for at least one matrix/layer or in the average;
4. the metrics table clearly shows LoRA within a few points of full fine-tuning while using less than 1% trainable parameters;
5. the router demo figure visually communicates "one general model, multiple LoRA experts."

If there are only 15 minutes left, stop coding and polish the slides.

17 Recommended Slide Order

1. **Motivation:** LoRA is cheap, but what geometry is it learning?
2. **Shortcut setup:** existing RoBERTa-base SST-2 full-FT and LoRA checkpoints.
3. **Full-FT SVD:** cumulative spectral energy plot.
4. **LoRA vs FT overlap:** subspace overlap plot.
5. **Performance table:** reported SST-2 accuracy and trainable parameter fraction.
6. **Mixture idea:** oracle-routed task LoRA pool.
7. **Takeaway:** update geometry gives a natural way to think about routing LoRA experts.
8. **Caveats:** fast prototype, one seed, checkpoint proxies, not a rigorous benchmark.

18 If Something Breaks

18.1 Model Loading Fails

Try:

```
python - <<'PY'
from transformers import AutoModelForSequenceClassification
for m in ["roberta-base", "rambodazimi/roberta-base-finetuned-FFT-SST2", "rambodazimi/roberta-
    base-finetuned-LoRA-SST2"]:
    print("loading", m)
    model = AutoModelForSequenceClassification.from_pretrained(m)
    print("ok", sum(p.numel() for p in model.parameters()))
PY
```

If the LoRA model fails, skip it and use any compatible RoBERTa-base LoRA checkpoint that loads.

18.2 SVD Is Slow

Only analyze layer 6:

```
roberta.encoder.layer.6.attention.self.query.weight
roberta.encoder.layer.6.attention.self.value.weight
```

This still gives a usable plot.

18.3 Overlap Looks Bad

Use these fallbacks in order:

1. Plot value matrices only.
2. Plot best layer only.
3. Plot top-1 to top-16 only.
4. Rename claim from "LoRA recovers full fine-tuning subspace" to "LoRA learns a distinct but comparable low-dimensional update."

18.4 Router Demo Is Too Much

Use a static diagram:

```
Input -> task router -> selected LoRA expert -> frozen RoBERTa-base
```

Then use model-card numbers for the task-specific experts.

19 Final Human Checklist

Before presenting, **Human** should check:

- Are all plots legible on slides?
- Are axes labeled with human-friendly names?
- Did we label proxy deltas honestly?
- Did we avoid saying the router was learned if it was oracle-routed?
- Did we state this is an exploratory prototype?
- Are the headline numbers big and easy to read?

20 Final Agent Checklist

Before handoff, **AI coding agent** should provide:

- all figures in `figures/`;
- all tables in `tables/`;
- a concise `README.md`;
- `presentation_snippets.md`;
- a single command or shell script to rerun everything;
- notes on any fallbacks used.

21 Sources to Keep Handy

Useful references and documentation:

- Hugging Face PEFT LoRA documentation: https://huggingface.co/docs/peft/package_reference/lora
- Hugging Face PEFT X-LoRA documentation: https://huggingface.co/docs/peft/package_reference/xlora
- Microsoft LoRA repository: https://github.com/microsoft/LoRA
- Microsoft LoRA NLU examples: https://github.com/microsoft/LoRA/blob/main/examples/NLU/README.md
- SST-2 LoRA checkpoint: https://huggingface.co/rambodazimi/roberta-base-finetuned-LoRA-SST2
- SST-2 full-FT checkpoint: https://huggingface.co/rambodazimi/roberta-base-finetuned-FFT-SST2